Does a Beta kernel beat the Gaussian in a small-budget 3DGS trainer?

George Yeo · CP4282 individual project · 13 November 2026

Summary

I replaced the Gaussian kernel in the course Warp 3DGS trainer with the Beta kernel from Deformable Beta Splatting, in which every splat learns one extra shape parameter b, and compared the two under a fixed protocol on the eight NeRF-synthetic scenes (10,000 splats, 400 px, 20,000 iterations).

- Beta is ahead by **+0.22 dB PSNR on average** over 18 seed-paired runs, with lower LPIPS in 15 of 18, at about 3% more time per iteration and one extra float per splat.
- The gain comes from **letting each splat learn b**. A Beta kernel with b frozen is about equal to the Gaussian (−0.03 dB).
- The gain shows up **at object boundaries**. Splats on thin structures and edges learn a sharper peak, and large splats on smooth surfaces learn a flatter, box-like profile.
- The gain is **not uniform across scenes**. Ficus (+0.65 dB) and lego (+0.42 dB) lead both under the protocol and from a full-capacity start. Other scenes are small, zero, or depend on the training setup.
- The protocol itself turned out to be noisy. Starting from 1,000 random splats without opacity reset leaves floaters that account for most of the seed-to-seed spread. Removing them changes the level by up to 0.9 dB but **does not change Beta's lead**.

The size of the kernel-only gain matches what the original paper reports in its own ablation (about 0.2 dB). The kernel is a real but small improvement; it is not where most of that paper's headline gain comes from.

1. Background and question

3D Gaussian Splatting [1] represents a scene as a set of anisotropic Gaussians and renders them by alpha blending. A Gaussian has one fixed profile: a smooth peak with long tails. A flat surface or a hard edge therefore needs many overlapping Gaussians to approximate.

Deformable Beta Splatting (DBS) [2] replaces the Gaussian with a bounded Beta kernel whose profile is controlled by a learnable parameter per splat. DBS reports a large gain over 3DGS-MCMC [3], but it changes three things at once: the kernel, the colour model and the densifier.

This project isolates the kernel. The questions are:

1. Inside the course trainer, with everything else held equal, does the Beta kernel beat the Gaussian?
2. If it does, is the gain from the kernel's shape or from learning b?
3. Where does it help (which scenes, which parts of the image), and why?
4. How reliable is the answer, given how much results move with the random seed?

2. Method

2.1 The kernel

In the course's notation [6], a splat's alpha at pixel p is $\alpha(p) = \alpha_0 \cdot w(p)$, where α_0 is the splat's opacity and w is the kernel weight. Write $q = (p - \mu)^\top \Sigma^{-1} (p - \mu)$ for the squared distance from the splat's projected centre,

measured with its projected covariance. The two kernels differ only in w:

- Gaussian: $w = \exp(-q / 2)$
- Beta: $w = (1 - q / 9)^{4 \cdot e^b}$ for $q < 9$, and 0 beyond

The 9 is 3σ squared, so the Beta kernel ends exactly at 3σ, and b is one extra learnable number per splat. At $b = 0$ the exponent is 4 and the profile is close to the Gaussian. A positive b gives a sharper peak; a negative b gives a flatter top with a steeper edge, approaching a box.

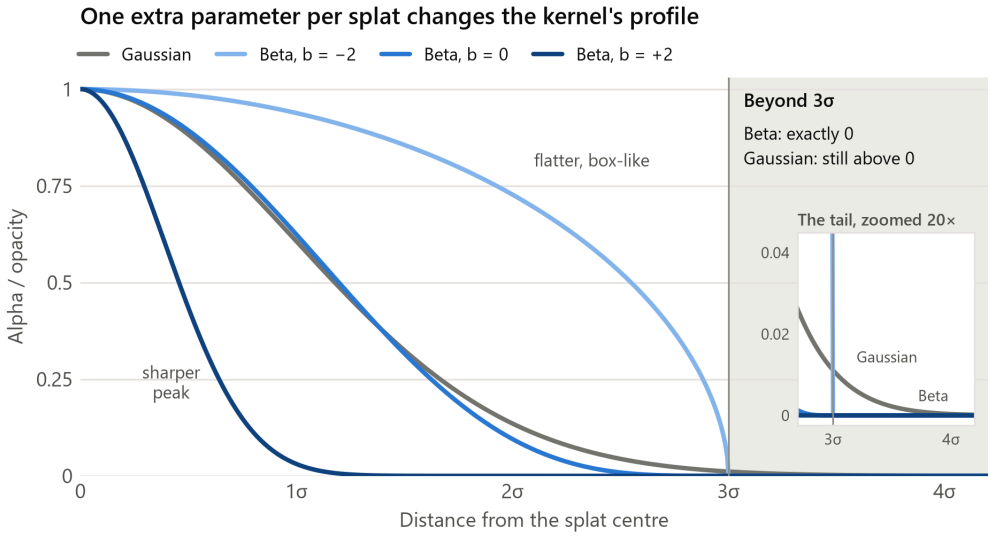


Figure 1. The Gaussian and the Beta kernel at three values of b, against distance from the splat centre.

b is clamped to [−2, 2]. The paper leaves b unbounded; the clamp is a safeguard I added, and widening it to [−4, 4] changes PSNR by at most 0.03 dB on lego and hotdog (one seed), so it does not affect the results.

2.2 Implementation

The kernel is implemented in the course trainer's forward and backward passes (`3dgs_trainer.py`), including the analytic gradient with respect to b. The paper gives the kernel but not its gradients, so I derived them: $\partial w/\partial q = -(\beta/9)(1 - q/9)^{\beta-1}$ and $\partial w/\partial b = w \cdot \beta \cdot \ln(1 - q/9)$, with $\beta = 4 \cdot e^b$. The main points:

- **b has its own Adam optimiser** and learning rate, and is projected back into the clamp after each step. Children created by densification inherit their parent's b.
- **Both kernels are truncated at the same alpha.** The course's Compact Box rule cuts a Gaussian where its alpha falls to $\sqrt{(\alpha_0 \cdot \alpha_{min})}$; the Beta kernel uses the same threshold, and the tile assignment uses the same per-splat support as shading. So neither kernel is drawn out to 3σ in this trainer: an opaque Gaussian is cut at 2.35σ and an opaque Beta splat with $b = 0$ at 2.12σ. The Beta kernel's bounded support therefore gives it no larger or smaller footprint for free.
- **Moment-matched initialisation.** A Beta splat at $b = 0$ is slightly narrower than a Gaussian with the same covariance. The initial scales are multiplied by a factor that matches the second moment of the truncated footprint at each splat's opacity (1.09 at $\alpha_0 = 1$). With it, a Gaussian model re-rendered as Beta agrees with the original to 44–46 dB, against 40 dB without scaling.
- **The Gaussian arm is unchanged.** Its gradients are bit-identical before and after the Beta code was added.

2.3 Correctness checks

No training run was counted until these passed:

- Finite-difference gradient checks for the Beta kernel, end to end and per pixel out to the rim, for b from −2 to 2 (relative error at most 1.2e-2).
- The sparse (sampled) loss path equals the dense path to 1e-7 for both kernels.
- A Beta arm with the kernel-aware densifier is bit-identical to the plain Beta arm at b = 0 through three densification events (on CPU; GPU runs are not bit-reproducible, see 4.5).

2.4 Arms

Arm	Kernel	b	Purpose
A	Gaussian	n/a	Baseline
B0	Beta	frozen at 0	Isolates the kernel shape
B	Beta	learnable per splat	The method under test
C	Beta	learnable	B plus a densifier that accounts for b when splitting

A, B0 and B share the same densifier and differ in one thing at a time: A → B0 changes the shape, B0 → B makes b learnable. Arm C is a side check, not part of the comparison. The course densifier ranks splats by gradient and decides between cloning and splitting by scale, both of which assume a Gaussian; arm C corrects both for b, to test whether the shared densifier holds Beta back.

---

3. Protocol

All settings were fixed before the main runs.

- **Data:** the eight NeRF-synthetic scenes [4], 100 training views, 200 test views, 400 × 400.
- **Training:** 20,000 iterations, random initialisation with 1,000 splats, the course densifier growing to a fixed capacity, opacity reset off.
- **Loss:** MSE over every pixel of the view. The course default samples 48 pixels per tile; the pilot found the dense loss more than 1 dB better for both arms.
- **Capacity: 10,000 splats for every scene.** A pilot on lego's validation split picked the smallest capacity whose Gaussian PSNR was within 0.5 dB of the largest one tried.
- **b learning rate: 0.0005,** chosen from three values on lego's validation split.
- **Choices on validation, results on test.** No test number was used to pick a setting.
- **Seeds:** two per scene and four on lego. Every Gaussian run has a Beta run with the same seed, and comparisons are made within the pair.
- **Metrics:** PSNR and LPIPS [5] on the test split.

Runs that depart from this protocol are labelled as side checks. Hardware: one RTX 5070, about 2.5 minutes of training per run.

The absolute numbers are far below published ones (lego is 27 dB here, 36 dB in the papers). That is the small budget, the low resolution and the random start, and it applies to both arms.

4. Results

4.1 Beta leads by 0.22 dB on average

Scene	Seeds	Gaussian (dB)	Beta (dB)	Beta − Gaussian	Per seed	LPIPS change
ficus	2	26.57	27.22	+0.65	+0.81, +0.49	−0.0052
hotdog	2	32.88	33.42	+0.53	+0.57, +0.50	−0.0056
lego	4	27.12	27.54	+0.42	+0.47, +0.18, +0.55, +0.47	−0.0025
drums	2	23.87	24.09	+0.22	+0.18, +0.26	−0.0068
mic	2	30.55	30.59	+0.04	−0.03, +0.11	−0.0022
materials	2	24.56	24.59	+0.03	+0.03, +0.04	−0.0045
ship	2	26.43	26.32	−0.10	−0.20, −0.00	+0.0026
chair	2	29.97	29.78	−0.19	−0.45, +0.08	+0.0003

Table 1. Test PSNR under the protocol (mean over seeds) and the paired difference.

- Over the 18 seed pairs the mean difference is +0.22 dB (standard deviation 0.32, standard error 0.075). Beta wins 14 pairs on PSNR and 15 on LPIPS.
- Counting each scene once, the mean is +0.20 dB with a standard error of 0.11 (six of eight scenes positive). So the lead on these scenes is clear, but eight scenes are too few to claim it for scenes in general.
- By the median test view, Beta is ahead in 17 of 18 pairs. Where it loses on the mean (ship, one chair seed), it wins most views by a little and loses a few badly (section 4.5).
- Cost: 7.1 ms per iteration against 6.9 ms, and one extra float per splat.

4.2 The gain comes from learning b

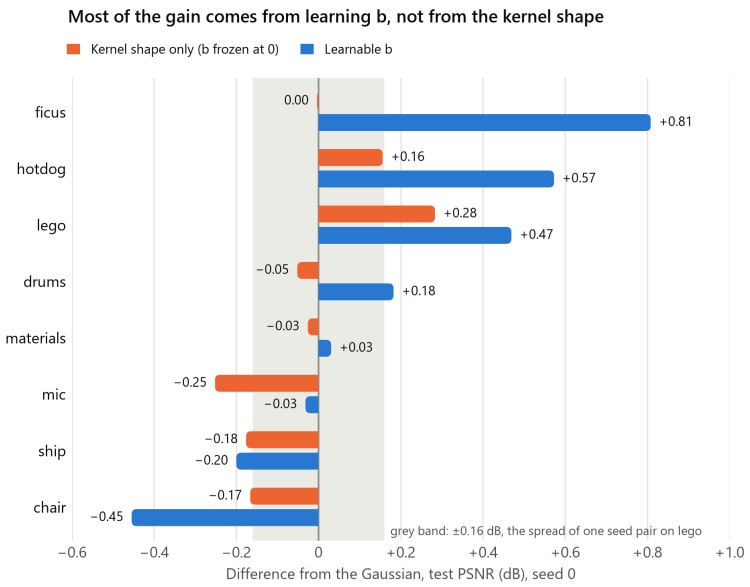


Figure 2. Frozen-b Beta (B0) and learnable-b Beta (B) against the Gaussian, seed 0.

Freezing b at 0 gives −0.03 dB on average over the eight scenes (standard error 0.06), which is inside the noise; it is ahead of the Gaussian on only two scenes. Unfreezing it adds +0.20 dB on average (standard error 0.11) and helps on six of the eight: +0.81 dB on ficus and +0.42 dB on hotdog. On lego the frozen

kernel already gains +0.28 dB, so the split there is unclear with one seed. On chair and ship both variants lose.

Those losses belong to the runs, not to the kernel. A second frozen-b seed on the three scenes where it lost gives B0 − A of −0.02 dB on mic, +0.05 dB on ship and +0.04 dB on chair (seed 0: −0.25, −0.18 and −0.17 dB). At seed 0 the views where a Beta arm is more than 0.5 dB behind come in runs of neighbouring test cameras (ship views 88–111, chair views 0–20 and 178–199), and the renders show stray splats there: under ship's basin and in front of the chair's seat. Outside those views the difference is +0.12 dB on ship and +0.02 dB on chair.

The kernel-aware densifier (arm C) changes nothing: 27.75 against 27.71 dB on lego and 27.47 against 27.50 dB on ficus (one seed each). So the shared densifier does not hold Beta back. DBS likewise uses a densifier that ignores the kernel's shape.

4.3 More splats do not shrink the lead

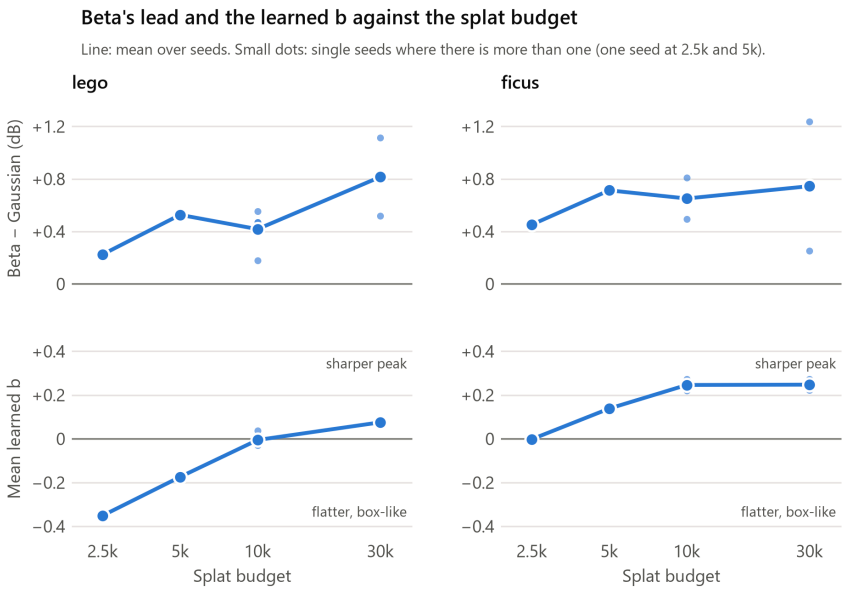


Figure 3. Top: Beta's lead against the splat budget. Bottom: the mean learned b.

I expected Beta to help most when splats are scarce, because one flexible splat could stand in for several Gaussians. The opposite holds. The lead is smallest at 2,500 splats and larger at every bigger budget on both scenes; between 5,000 and 10,000 it does not change by more than the noise. At 30,000 (a side check) it roughly doubles on lego and stays level on ficus, though the two seeds disagree widely there.

The learned b explains part of this. With few splats, b drifts negative on lego: splats flatten to cover area. As the budget grows, b rises: splats can afford to sharpen. The shape freedom is used more when there are enough splats to specialise.

4.4 What b does: flat surfaces and sharp edges

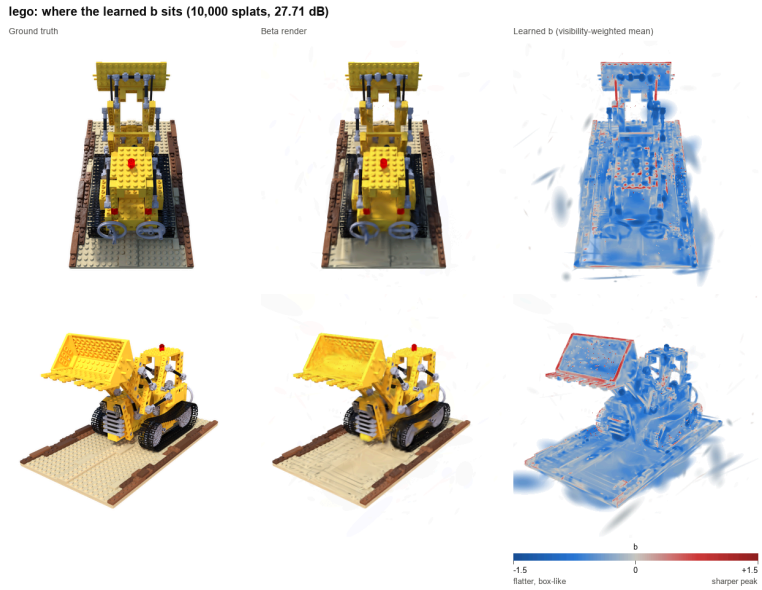


Figure 4. Lego, two test views: ground truth, Beta render, and a map of the mean b seen at each pixel (blue = flat, grey = 0, red = sharp).

- Broad surfaces (the baseplate, body panels) are covered by flat splats. Rims, thin rods and stud outlines are drawn by sharp ones. On ficus the leaves and stems are sharp and the pot is flat.
- In all eight scenes b is negatively rank-correlated with splat size (−0.42 to −0.69): large splats flatten, small splats sharpen.

To see where the error falls, I split every test pixel into four regions using the ground truth only: background, a silhouette band within 2 px of the object's outline, high-gradient detail inside the object, and smooth interior.

- **Silhouette error falls in seven of eight scenes** (by 15% on ficus and 17% on lego) and is flat on the eighth.
- On ficus, 78% of the Gaussian's error sits in the silhouette band (thin leaves against a white background), and almost all of Beta's gain is there.
- The learnable b is what does it: against the frozen kernel, learnable b lowers silhouette error on seven of the eight scenes (not ship), while the frozen kernel against the Gaussian has no consistent effect.

So the mechanism is consistent across the evidence: a bounded kernel with a learnable profile draws boundaries with less blur than a Gaussian's tails allow.

4.5 Why the seed matters, and how to reduce it

Under the protocol, the Gaussian's PSNR on lego varies by 0.30 dB (standard deviation) across four seeds. That is as large as most of the per-scene differences in Table 1, so I looked into where it comes from.

- **It is not independent noise.** The seed-specific error is strongly correlated between neighbouring test views (0.95 at one view apart). Defects span about ten adjacent cameras.
- **It is floaters.** The worst seed loses more than 1 dB on the same views in both arms, and the renders show the same white streaks in both. A few stray splats survive in places the training cameras constrain poorly.
- **Part of it is chance.** GPU float atomics make runs non-reproducible. The same seed run four times varies by 0.12 dB, and one repeat produced the same floater that had looked like a property of another seed.

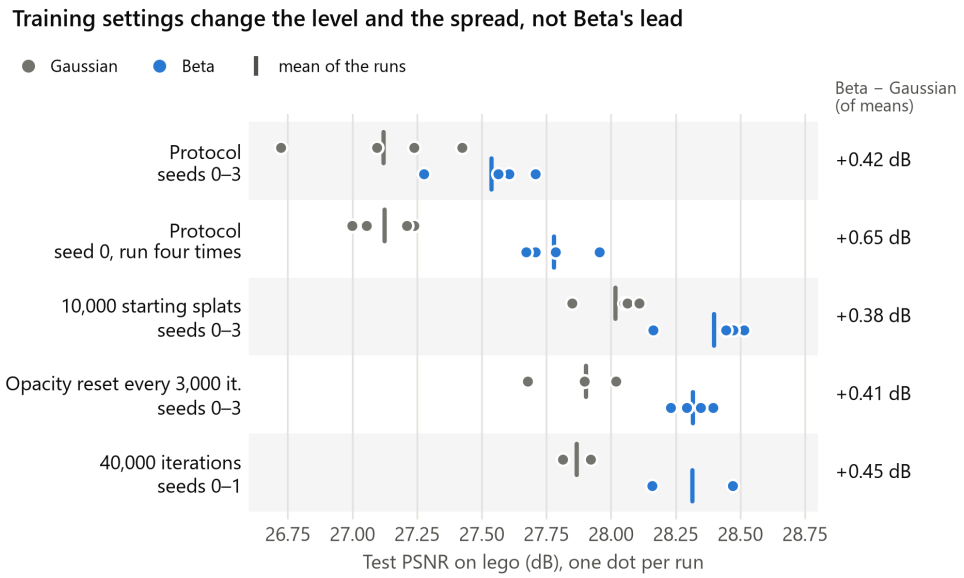


Figure 5. Every lego run under five training settings.

Two changes remove most of the spread on lego (side checks):

Setting (lego, 4 seeds)	Gaussian	Beta	Beta − Gaussian
Protocol	27.12 ± 0.30	27.54 ± 0.19	+0.42
Start from 10,000 splats	28.02 ± 0.12	28.40 ± 0.16	+0.38
Opacity reset every 3,000 iterations	27.90 ± 0.16	28.32 ± 0.07	+0.41

Table 2. Mean ± standard deviation over seeds.

- With opacity reset, no seed has a view more than 1 dB below the others, and the seed-by-view variance falls about tenfold. This is the direct evidence that floaters cause the spread.
- Both changes also raise quality by 0.8–0.9 dB on lego. I had left opacity reset off expecting it to interfere with pruning; on lego it helps instead.
- **Beta's lead is the same in every row**, and also at 40,000 iterations (+0.45 dB) and at 200 px and 800 px (+0.49 and +0.66 dB, two seeds each).

For a comparison like this one, the practical lessons are to pair runs by seed (which halves the noise of the difference, from 0.31 to 0.16 dB), and to prefer a training setup that does not leave floaters.

4.6 All eight scenes from a full start

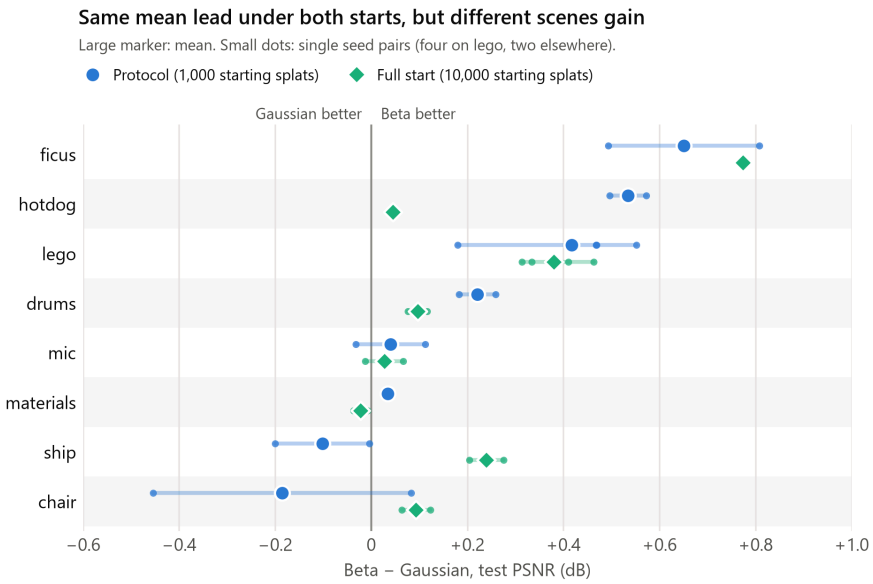


Figure 6. Beta's lead per scene under the protocol (blue) and from 10,000 starting splats (green).

Because the 1,000-splat start is a source of noise, I repeated the whole comparison from 10,000 starting splats (a side check, same seeds).

- **The mean lead is identical: +0.22 dB** over the same 18 pairs, with Beta winning 15 on PSNR and 16 on LPIPS.
- **The lead is far more repeatable.** A scene's two seeds disagree on it by 0.04 dB on average, against 0.21 dB under the protocol.
- **Ship and chair turn positive** (−0.10 → +0.24 dB and −0.19 → +0.09 dB). Their losses under the protocol were floaters: on ship, Beta was better inside the object and much worse in the background, and from a full start that background error is gone.
- **Hotdog's lead almost disappears** (+0.53 → +0.05 dB). The full start helps the Gaussian by 1.1 dB there and Beta by only 0.65 dB. I did not expect this and cannot fully explain it; floaters account for part of the drop.
- **Ficus and lego keep their lead** (+0.77 and +0.38 dB).
- Silhouette error again falls in seven scenes and is flat in the eighth.

A full start is not better everywhere: both arms lose about 0.7 dB on ficus and 0.4 dB on mic. It also does not remove every floater. On chair's worse seed the views where Beta is more than 0.5 dB behind fall from 78 to 14 of 200, not to zero, and hotdog and chair still show runs of bad views in both arms. It is a robustness check, not a better protocol.

4.7 Fewer training views

With 50 training views instead of 100 (seed 0), Beta's lead holds on lego (+0.81 dB) and ficus (+0.85 dB). With 25 views lego keeps a lead (+0.38 dB) but ficus loses 1.27 dB; both ficus runs overfit badly in that regime. This is one seed, so I report it as an anomaly, not a finding.

5. Comparison with the paper

	DBS paper [2]	This project
Kernel-only gain	+0.16 dB (Tanks and Temples), +0.21 dB (Deep Blending); Table 3, both arms with MCMC densification and SH colour	+0.22 dB (NeRF-synthetic)
Full method on NeRF-synthetic	+0.86 dB over 3DGS-MCMC (33.80 → 34.66)	not tested
b learning rate	0.001	0.0005
Scenes that gain most	ficus, hotdog, lego	ficus, lego (hotdog under the protocol only)
Densifier	kernel-agnostic MCMC	a kernel-aware densifier adds nothing

The kernel-only gain agrees. Most of the paper's headline improvement comes from its colour model and densifier, not from the kernel. Two things here are not in the paper: the frozen-b ablation, which shows the shape alone does nothing, and the statistics of the learned b.

6. Limitations

- Two seeds per scene (four on lego); one seed for the ablation (a second seed on three scenes only), the densifier, the low-budget points and the view-count sweep.
- From the 1,000-splat start Beta leaves more seed-dependent bad views than the Gaussian (218 against 149 of 1,600). I did not find out why.
- Eight synthetic scenes, random initialisation, no real captures. Counting scenes as the unit, the mean lead is about twice its standard error.
- One capacity for the main table. The budget trend rests on lego and ficus.
- Which scenes gain depends on the starting splat count. Only ficus and lego lead in both settings.
- The seed-variance experiments (same-seed repeats, opacity reset, iterations, resolution) are on lego only.
- b's learning rate was tuned on one scene, and the three values tried differ by less than the seed noise.
- The link between b and the gain is a consistent pattern across runs, not a controlled test.

7. What I would do differently with more time

- **Choose a quieter protocol.** I froze a 1,000-splat start with opacity reset off before I knew they were the main source of seed noise. I would test opacity reset on all eight scenes first and, if it holds up as it did on lego, make it part of the protocol.
- **More seeds where the claims are thinnest.** The frozen-b ablation is one seed per scene. It is the result that is new relative to the paper, so it deserves at least the two seeds the main comparison has, and four on a second scene besides lego.
- **Test the mechanism, not only the outcome.** That floaters grow from a sparse start, and that b helps at boundaries, are both patterns consistent with the runs. I would trace floaters back to the densification step that created them, and intervene on b directly (for example freeze b for large splats only) to see whether the boundary gain follows.
- **Explain the two open cases.** Why hotdog's lead depends on the start, and why Beta leaves more floaters than the Gaussian from a sparse start.

- **Leave the synthetic, fixed-colour setting.** A point-cloud initialisation, view-dependent colour, a larger budget and real captures would show whether a 0.2 dB kernel gain survives once the other limits on quality are lifted.

---

8. Conclusion

In this trainer, at a small splat budget, the Beta kernel with a learnable per-splat shape is a small, real improvement over the Gaussian: about 0.2 dB on average, at almost no cost. On lego the lead held under every change in training setup I tried, and the eight-scene mean was the same from a different start. It is not marginally useless, and it is not the large gain the DBS headline numbers suggest. The benefit is specific. It comes from the learnable b, it appears at object boundaries and thin structures, and it is established on two scenes (ficus and lego), not on all eight.

The second result is about method. A third of a decibel of seed noise can hide or fake an effect of this size. Pairing by seed, looking at per-view errors and renders instead of means, and re-testing the conclusion under a changed setup were what separated the kernel's effect from the floaters.

---

9. Code and reproduction

Repository: https://github.com/george-yeo/cp4282-project

The repository's `README.md` has the steps. In short:

- **Without a GPU.** `results/` holds the result files of all 154 runs: settings, summary metrics, per-view test metrics, training logs and the learned b of every splat. `python scripts/report_figures.py --root results` redraws the figures from them, each with a CSV of its plotted values, so every number here can be traced to its runs.
- **Environment:** Python with the packages in `requirements.txt` (Warp 1.16, NumPy, Pillow, PyYAML, plyfile; torch and lpips for LPIPS). All runs were on one RTX 5070.
- **Data:** the eight NeRF-synthetic scenes [4] in one folder.
- **Checks:** `checks/check_beta_gradients.py`, `checks/check_sparse_matches_dense.py` and `checks/check_arm_equivalences.py`, plus the course's `3dgs_gradient_check_gpu.py`.
- **All runs:** `bash scripts/reproduce.sh <data folder>` trains and evaluates every run in the order it was made: the protocol runs and each side check in its own folder. It is about 6.5 hours of training plus evaluation, it is resumable, and the script itself is the list of exact commands. By default it uses the frozen protocol choices; `--repick` reruns the rules that chose them.

Training on a GPU is not bit-reproducible (section 4.5): the same seed run four times varied by 0.12 dB. A rerun should match these numbers to within a few tenths of a dB per run, not exactly.

---

10. AI use declaration

Tool. Claude Code (Anthropic), a coding agent, across the whole project.

What I used it for.

- Implementing the Beta kernel, its gradients, the frozen-b and densifier arms and the run driver on top of the course trainer. Every commit after my assignment 2 submission was made with it and carries a co-author line.
- Running the training queues and side checks, and collecting their results.

- Analysing the results: the per-view and per-region breakdowns, the seed-variance study and the figures.
- Drafting this report, the analysis and investigation notes, and the presentation slides.

What was mine. The research question and its scope (change the kernel and nothing else), the rules the work had to follow (leave the course kernels and checks unmodified, choose settings on validation and never on test, freeze the protocol before the main runs, keep off-protocol runs separate and labelled), the decisions on which follow-up experiments to run, and the review of the results and of the explanations offered for them.

How I checked its output.

- **Code:** finite-difference gradient checks for the new kernel, a check that the sampled and dense loss paths agree, and a check that the Gaussian arm's gradients are bit-identical before and after the change (sections 2.2 and 2.3). No run was counted before these passed.
- **Numbers:** every figure and table is generated by a script from the run folders, with a CSV of the plotted values, so each number can be traced to its runs.
- **Claims:** predictions were written down before each batch of runs, and `INVESTIGATION_BETA.md` lists the claims that later runs overturned (for example that hotdog was a scene that suits Beta, and that opacity reset would cost quality).
- **The paper's numbers** were re-read from the arXiv page before being quoted.
- **Renders** were inspected directly wherever a mean looked surprising, which is how the floaters were found.

References

1. B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM Transactions on Graphics (SIGGRAPH), 2023.
2. R. Liu, D. Sun, M. Chen, Y. Wang, A. Feng. Deformable Beta Splatting. SIGGRAPH 2025. arXiv:2501.18630.
3. S. Kheradmand et al. 3D Gaussian Splatting as Markov Chain Monte Carlo. NeurIPS 2024.
4. B. Mildenhall et al. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. ECCV 2020.
5. R. Zhang et al. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. CVPR 2018.
6. CP4282 course notes and assignment 2 trainer (the Warp 3DGS trainer this project builds on, including its Compact Box rule). https://weitsang.github.io/gaussian-splatting-notes/

Appendix: where the details are

- `README_BETA.md`: code changes, design decisions, the checks and how to run the queues.
- `ANALYSIS_BETA.md`: every result with its table (19 points), including the ones summarised here.
- `INVESTIGATION_BETA.md`: the order in which the findings were reached, the predictions written before each batch of runs, and the claims that were corrected along the way.
- `figures/report/`: the figures above as PDF, with a CSV of the plotted values, made by `scripts/report_figures.py`.